GitHub Practical Report –Scientific Programming

Ioannis Vakkas

1. Introduction

This report covers the GitHub practical on scientific programming, which I missed and completed individually afterwards. All seven activities were reproduced in a real Git repository (10 branches, 9 merged pull requests, one resolved merge conflict), ending in a reproducible Python pipeline on `main`.

Repository: github.com/ioannis13vak/wine-analysis (all branches and the full commit history).

Deviations from the group setup. I took all four roles (cleaning, statistics, visualization, PCA), each on its own branch, and reviewed each pull request myself. During development a bare Git repository acted as the remote (clone, push and pull work exactly as with GitHub), and pull requests were done as reviewed merges named "Merge pull request #n". The finished repository, with all branches and history, was then pushed to GitHub, where the report itself was added through a real GitHub pull request. The analysis uses Python (pandas, NumPy, SciPy, scikit-learn, matplotlib).

Data. I used the UCI **Wine dataset** (Aeberhard & Forina, 1992): 178 Italian wines from 3 cultivars (the classes), 13 chemical variables. Because it has no errors, I added some on purpose so the cleaning step had work to do: 15 missing values, 3 impossible values and 2 duplicate rows (fixed random seed). Exact figures may therefore differ slightly from the course file; methods and conclusions do not.

2. Workflow used

Every change followed the same cycle: `git switch -c <branch>` → edit → `git add` + `git commit -m "<clear message>"` → `git push -u origin <branch>` → pull request → review and fixes → merge into `main` → `git pull` on the other copies. Nothing was committed directly to `main`, and commit messages describe the change ("Add missing value handling"), never "update".

3. Activity 1 – Repository setup

The repository `wine-analysis` was created with the required structure (`data/wine_data.csv`, `src/` with one script per component, `results/`, `README.md`, `.gitignore`). After cloning, the first workflow was completed on branch `readme-setup`: a contributor list was added to the README, committed ("Add contributor list to README"), pushed, and merged as PR #1.

4. Activity 2 – Data loading and cleaning

The cleaning script (`src/data_cleaning.py`, branch `data-cleaning`) reduced the raw file from 180 to 178 rows and imputed 18 values (0.78% of all cells), producing `data/wine_clean.csv` with no missing values.

The raw file had 180 rows × 14 columns (13 numeric variables + class).

Problem found	Action taken
2 duplicate rows	Removed (180 → 178 rows)
15 missing values	Imputed
Alcohol = −13.2, magnesium = 1270, ash = 0	Flagged by range checks, set to missing, imputed

Plausible ranges are defined at the top of the script. Missing values were filled with the **median of the same class**: robust to outliers, and it preserves the differences between cultivars. Less than 1% of cells were affected. The script and the cleaned file were committed, pushed and opened as pull request #2 (see section 7).

5. Activity 3 – Exploratory data analysis

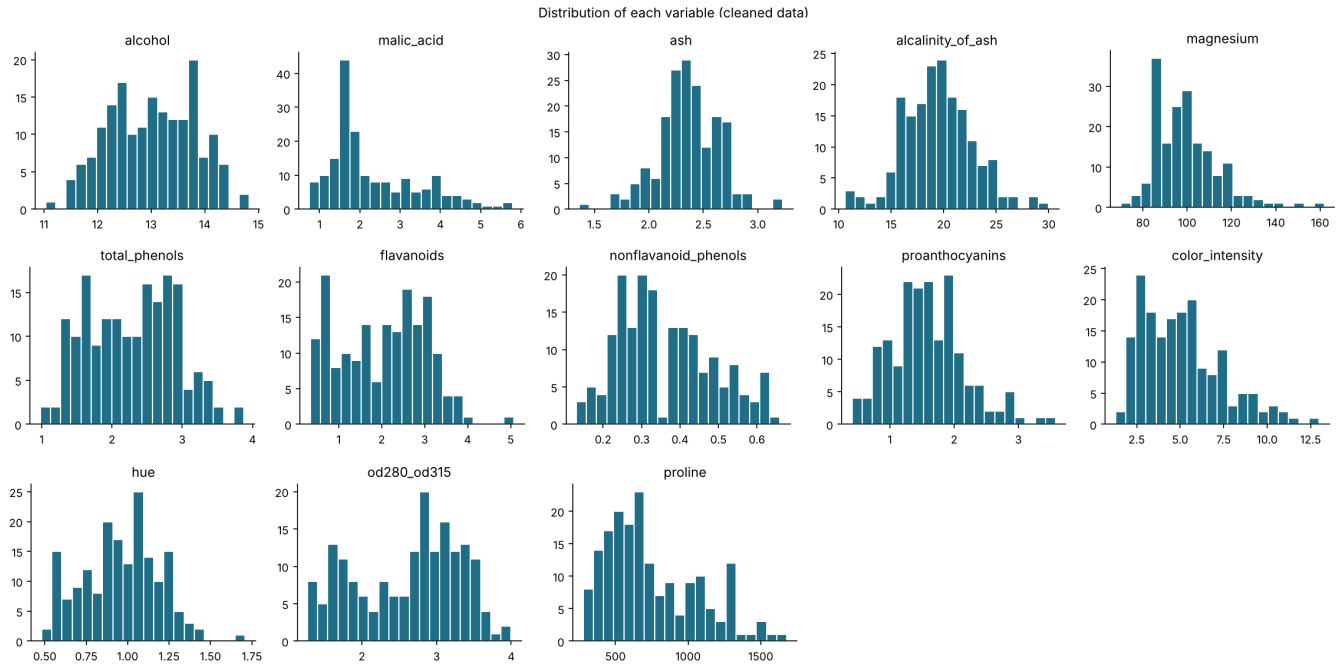
All 13 variables differ significantly between the three wine classes (Kruskal–Wallis, every $p < 0.00001$); flavanoids, proline, colour intensity and alcohol separate the classes most clearly. The work was split over two branches: `descriptive-stats` (`src/descriptive_stats.py`) and `visualization` (`src/visualization.py`). All tables and figures are written to `results/`.

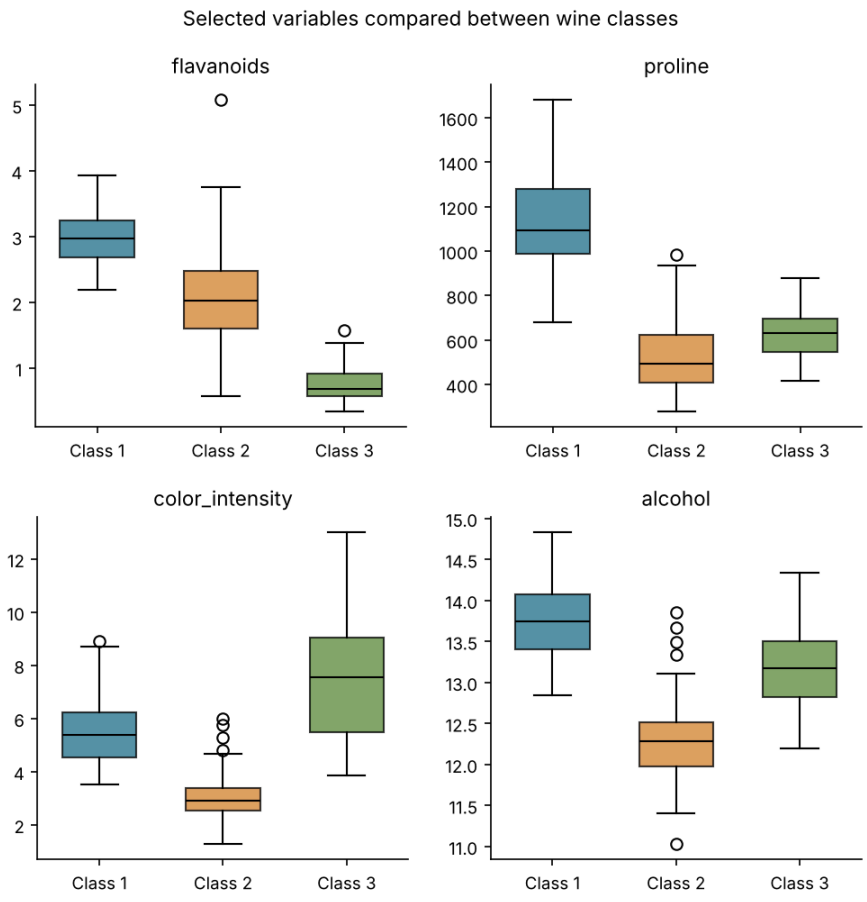
Mean, median, SD, minimum and maximum of every variable are in the Appendix; histograms of all variables are shown below. The key point for later: the variables are on very different scales (SD of proline 315, of hue 0.23), and malic acid, colour intensity and proline are right-skewed.

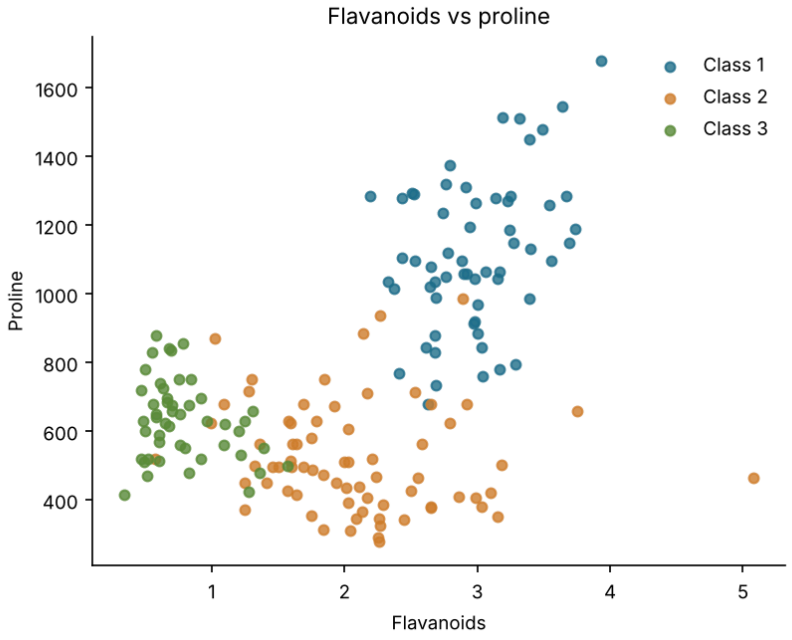
Comparison between classes

Variable	Class 1 mean	Class 2 mean	Class 3 mean	Kruskal–Wallis H
Flavanoids	2.98	2.08	0.78	130.5
Proline	1116.7	519.7	630.9	119.2
Colour intensity	5.53	3.08	7.42	116.7
Alcohol (%)	13.74	12.29	13.17	110.6
OD280/OD315	3.16	2.79	1.68	107.1
Ash (weakest)	2.46	2.24	2.44	26.6

Class 1 has the highest proline and alcohol, class 3 the lowest flavanoids and the darkest colour, and class 2 the lowest colour intensity and alcohol. The Kruskal–Wallis test was chosen because several variables are skewed, so it does not assume normality.







The scatter plot shows that just two variables, flavanoids and proline, already separate the three classes almost completely.

6. Activity 4 – Scaling and PCA

Scaling decides what PCA sees: log transformation and autoscaling both separate the three classes well (silhouette ≈ 0.53), while Pareto scaling on this data is dominated by proline and separates only class 1 (silhouette 0.20). The analysis is in `src/pca_analysis.py` on branch `pca-analysis`.

Methods

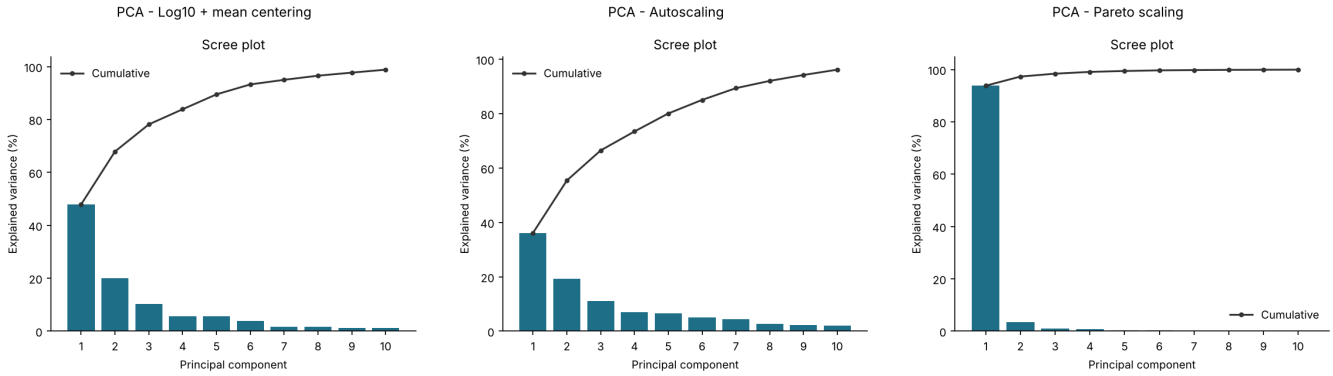
For a variable x with mean $\bar{x}$ and standard deviation s:

- **Log + mean centering:** $x' = \log_{10}(x) - \text{mean}(\log_{10} x)$
- **Autoscaling:** $x' = (x - \bar{x}) / s$
- **Pareto scaling:** $x' = (x - \bar{x}) / \sqrt{s}$

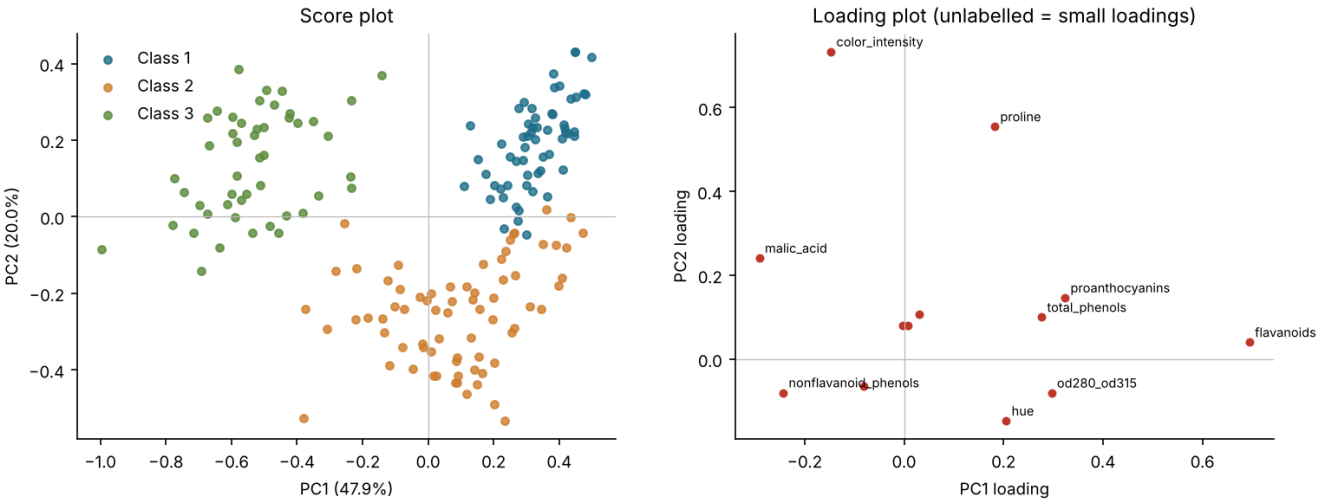
Autoscaling makes every variable count equally; Pareto scaling shrinks large variables less; the log transformation compresses large values and reduces skewness. For each model a scree plot, a score plot (PC1 vs PC2) and a loading plot were made, and class separation was measured with the silhouette score (−1 = full overlap, 1 = perfectly separated).

Comparison of the three models

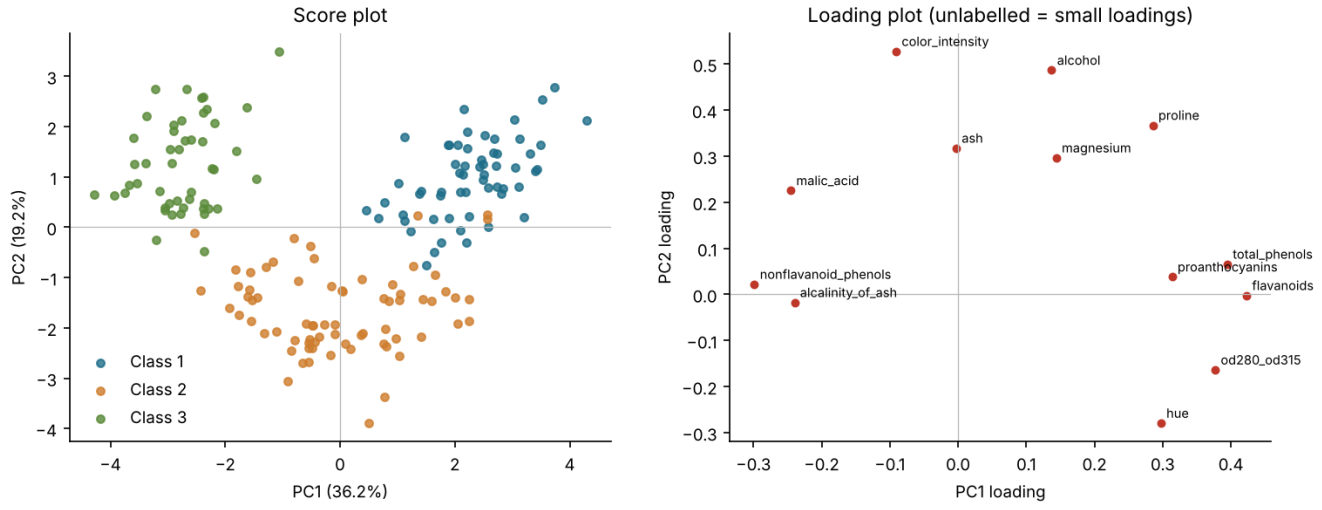
Scaling	PC1 (%)	PC2 (%)	PC1+PC2 (%)	Silhouette	Strongest on PC1	Strongest on PC2
Log + mean centering	47.9	20.0	67.9	0.53	Flavanoids, proanthocyanins, OD280/OD315	Colour intensity, proline, malic acid
Autoscaling	36.2	19.2	55.4	0.53	Flavanoids, total phenols, OD280/OD315	Colour intensity, alcohol, proline
Pareto scaling	93.8	3.5	97.3	0.20	Proline, magnesium, alcalinity of ash	Magnesium, proline, alcalinity of ash

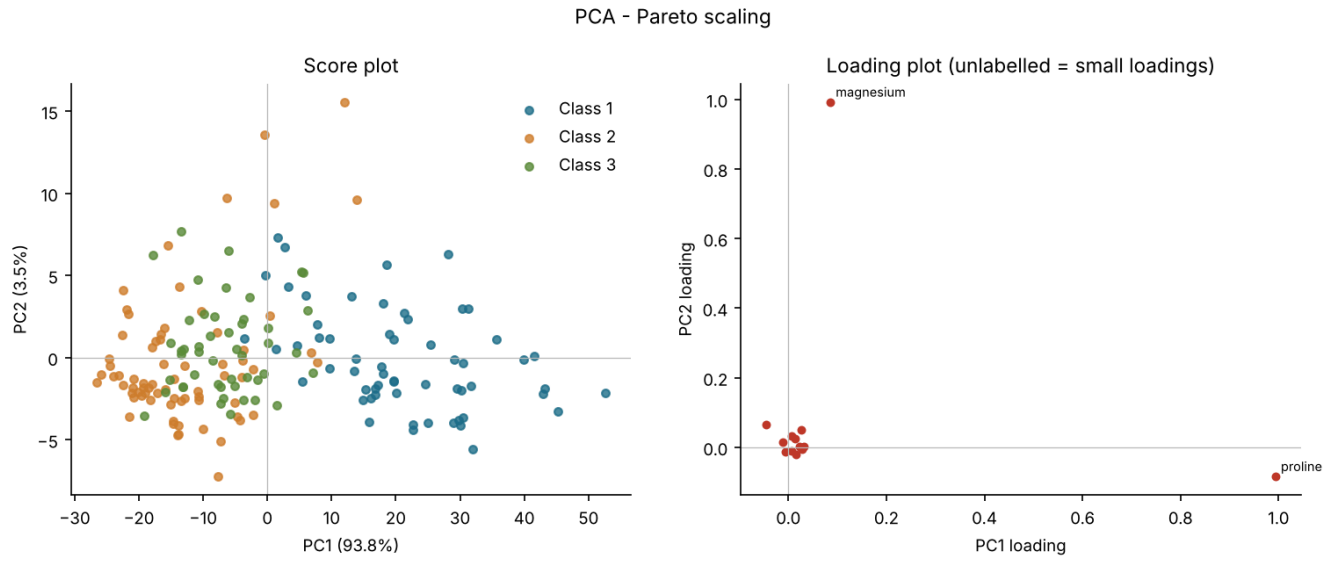


PCA - Log10 + mean centering



PCA - Autoscaling





Discussion

How scaling affects the results. PCA looks for directions of largest variance, so it follows whichever variables have the largest numbers. Under Pareto scaling, proline (SD 315) still has a variance of about 315 after dividing by $\sqrt{315}$, against about 14 for magnesium and below 3.5 for every other variable. PC1 therefore explains 93.8% of the variance but is almost a copy of proline (loading ≈ 1.0); the high explained variance is an artefact of units, not real structure. The log transformation and autoscaling remove this unit effect, so the variance spreads over several components and PC1 drops to 48% and 36%.

Separation between classes. With log and autoscaling, the score plots show three clear groups: class 1 and class 3 at opposite ends of PC1 and class 2 below them on PC2 (silhouette 0.53 for both). With Pareto scaling, class 1 (high proline) separates along PC1, but classes 2 and 3 overlap because their proline levels are similar (520 vs 631).

Variables that drive the first components. In the two well-scaled models, PC1 is a phenolic axis: flavanoids, total phenols, proanthocyanins and OD280/OD315 load positively and non-flavanoid phenols and malic acid negatively, so it runs from class 1 (rich in phenols) to class 3 (poor). PC2 is driven by colour intensity, proline and alcohol, which picks out class 2 (light, low alcohol). This agrees with the class comparison in section 5.

Conclusion. For chemical data measured in different units, autoscaling or a log transformation should be used. Pareto scaling is common in metabolomics, where variables share a unit, but it is not suitable here without first log-transforming proline.

7. Activity 5 – Pull requests and code review

Each analysis branch went through a pull request with at least one review comment, and every comment led to a follow-up commit before the merge. Working alone, I wrote the reviews myself after finishing each component, reading the code as a teammate would.

PR	Branch	Review comment	Author's response (commit)
#2	data-cleaning	"Could you report how many values were imputed in total, so the reader knows the impact on the data?"	Added a count and percentage of imputed cells: 18 values, 0.78% (`9da3182` Report total number of imputed values)
#3	descriptive-stats	"The task asks to compare variables between classes. Can a statistical test support the class means?"	Added a Kruskal–Wallis test per variable, chosen because several variables are skewed (`a283dbb` Add Kruskal-Wallis test comparing classes)
#4	visualization	"The orange median line is invisible on the orange class-2 boxes."	Medians drawn in black for contrast (`b8e9335` Draw boxplot medians in black)
#5	pca-analysis	"In the Pareto loading plot the labels overlap and cannot be read."	Only the 8 strongest loadings were labelled; this still overlapped near zero, so a follow-up PR #9 labels only variables with a loading of at least 20% of the largest (threshold set at the top of the script) (`43c2297` Label only strongest loadings)

Each fix was committed to the same branch and pushed, which updates the open pull request, the PR was merged only after that. The reviews improved the reporting of the cleaning step and caught two figures that were hard to read.

8. Activity 6 – Merge conflict

Two branches from the same `main` wrote different text under `## Project description` in README.md. Branch A was merged first (PR #6). When branch B merged `main` into itself, Git reported `CONFLICT (content): Merge conflict in README.md` and marked the lines:

```
<<<<<<< HEAD
Chemical analysis of 178 Italian wines from three cultivars using PCA.
=======
This project builds a reproducible analysis pipeline for the wine dataset.
>>>>>>> main
```

The upper part is branch B, the lower part is what was already on `main`. The two sentences complement each other, so both were kept in one description. The markers were removed, the file committed ("Resolve README conflict: combine both project descriptions") and pushed, and PR #7 then merged without conflict. Resolving on the feature branch keeps `main` working at all times.

9. Activity 7 – Final repository

`main` holds the complete pipeline, and a fresh clone reproduces every result with one command: running `python run_all.py` in a new clone regenerated all 13 result files and left `git status` empty, i.e. the outputs were byte-identical to the committed ones.

Pipeline: raw data → data cleaning → clean dataset → exploratory analysis → scaling → PCA → figures/results.

Each stage has its own script in `src/` (cleaning, descriptive statistics, visualization, PCA), and `run_all.py` runs them in order; all figures and tables land in `results/`.

The README explains the purpose, the repository structure, how to run the analysis (`pip install -r requirements.txt`, then `python run_all.py`), the required packages and who contributed each component.

Pulling changes made by others. A second clone, standing in for a teammate's computer still at PR #5, ran `git pull` and was fast-forwarded to the current `main` (`7cba2f9..b65b5e9`), receiving the README, `requirements.txt` and `run_all.py` changes from PRs #6–#8.

10. Conclusion

Scientific result. The three wine classes differ in all 13 variables, most clearly in flavanoids, proline, colour intensity and alcohol. PCA shows this only when variables are on a comparable scale (log or autoscaling); Pareto scaling on mixed units lets proline dominate.

What I learned. Branches let several people work in parallel without breaking `main`; clear, small commits make the history readable; review improves even working code; and a merge conflict is simply a question Git cannot answer alone. A repository is reproducible only with data, code, dependencies and run instructions together.

Sources

- Aeberhard, S. & Forina, M. (1992). *Wine* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5PC7J
- scikit-learn: `sklearn.datasets.load_wine`, Python copy of the UCI Wine dataset.

Appendix – Descriptive statistics (cleaned data)

Variable	Mean	Median	SD	Min	Max
Alcohol (%)	13.01	13.05	0.81	11.03	14.83
Malic acid	2.34	1.87	1.12	0.74	5.80
Ash	2.36	2.36	0.27	1.36	3.23
Alcalinity of ash	19.50	19.50	3.34	10.60	30.00
Magnesium (mg/L)	99.52	98.00	14.07	70.00	162.00
Total phenols	2.30	2.36	0.63	0.98	3.88
Flavanoids	2.03	2.14	1.00	0.34	5.08
Non-flavanoid phenols	0.36	0.34	0.12	0.13	0.66
Proanthocyanins	1.59	1.56	0.57	0.41	3.58
Colour intensity	5.07	4.75	2.31	1.28	13.00
Hue	0.96	0.97	0.23	0.48	1.71
OD280/OD315	2.61	2.78	0.71	1.27	4.00
Proline	747.6	673.5	315.1	278	1680